Cloud et systèmes distribués

Première fiche de compréhension • Master DSICS • 23 octobre 2026

Le point de départ du cours : faire coopérer plusieurs machines pour fournir un service utile, tout en maîtrisant les délais, les pannes et les copies de données. Le cloud est une façon de mettre ces ressources à disposition.

Le livre qui fonde le cours

Maarten van Steen et Andrew S. Tanenbaum, Distributed Systems, 4e édition, 2023. Le professeur le désigne explicitement page 3, puis dans la bibliographie page 41. Les deux autres livres cités sont des références complémentaires.

Page officielle du livre et des diapositives • Demander la copie numérique gratuite. Le téléchargement personnalisé demande une adresse mail et un CAPTCHA. La page officielle annonce la révision 4.03, publiée en janvier 2025.

Résumé du livre par thème

Aperçu du programme du livre, vérifié dans les diapositives officielles ; il ne remplace pas une lecture intégrale. Les chapitres 1 et 2 correspondent au périmètre principal du PDF fourni.

Chapitre	Question centrale à comprendre
1 Introduction	Pourquoi distribuer un service, et quels compromis accepter ?
2 Architectures	Comment organiser composants, couches, services et pairs ?
3 Processus	Comment exécuter le travail avec processus, threads et virtualisation ?
4 Communication	Comment les composants échangent-ils appels et messages ?
5 Coordination	Comment ordonner et coordonner les actions sans horloge globale parfaite ?
6 Nommage	Comment retrouver une ressource à partir de son nom ?
7 Cohérence et réplication	Comment gérer plusieurs copies qui évoluent ?
8 Tolérance aux pannes	Comment maintenir ou rétablir le service malgré les défaillances ?
9 Sécurité	Comment protéger les échanges et contrôler les accès ?

Ordre de travail conseillé : définitions → transparence → scalabilité et files d'attente → architectures → routage P2P. Ensuite seulement, approfondir les autres chapitres du livre.

Plusieurs nœuds et un service

Support du cours : pages 11 à 15 et 19

Un **système distribué** fait coopérer des nœuds autonomes et présente, autant que possible, un service cohérent à ses utilisateurs. Un nœud peut être une machine ou un processus logiciel. Chacun exécute ses propres actions et peut tomber en panne séparément.



Distribué ne veut pas dire décentralisé. Plusieurs serveurs peuvent dépendre d'une autorité centrale. Décentraliser consiste à répartir aussi le contrôle. Les dessins de topologie seuls ne suffisent pas à définir cette distinction.

Deux difficultés fondamentales : les nœuds n'ont pas de temps global parfaitement partagé ; une absence de réponse ne permet pas, à elle seule, de savoir si un serveur est lent, inaccessible ou arrêté.

Les quatre objectifs de conception

Partager les ressources ; **masquer** les détails de distribution utiles à masquer ; **ouvrir** le système grâce à des interfaces interopérables ; **passer à l'échelle** sans dégrader excessivement le service. « Ouvert » ne signifie pas accessible sans authentification.

Les sept formes de transparence

Forme	Ce que l'utilisateur ne doit pas avoir à gérer
Accès	Les formats et méthodes d'accès différents.
Localisation	L'emplacement exact de la ressource.
Migration	Le changement d'emplacement d'une ressource.
Relocalisation	Le déplacement pendant que la ressource est utilisée.
Réplication	L'existence de plusieurs copies.
Concurrence	Les accès simultanés de plusieurs utilisateurs.
Panne	La défaillance et la reprise, dans les limites possibles.

Exemple : tu ouvres un fichier sans choisir le serveur qui le stocke. Cette transparence a un coût : copies à mettre à jour, coordination et délais. Un service « cohérent » vu par l'utilisateur ne garantit pas automatiquement une cohérence forte de toutes les données.

Comprendre la saturation

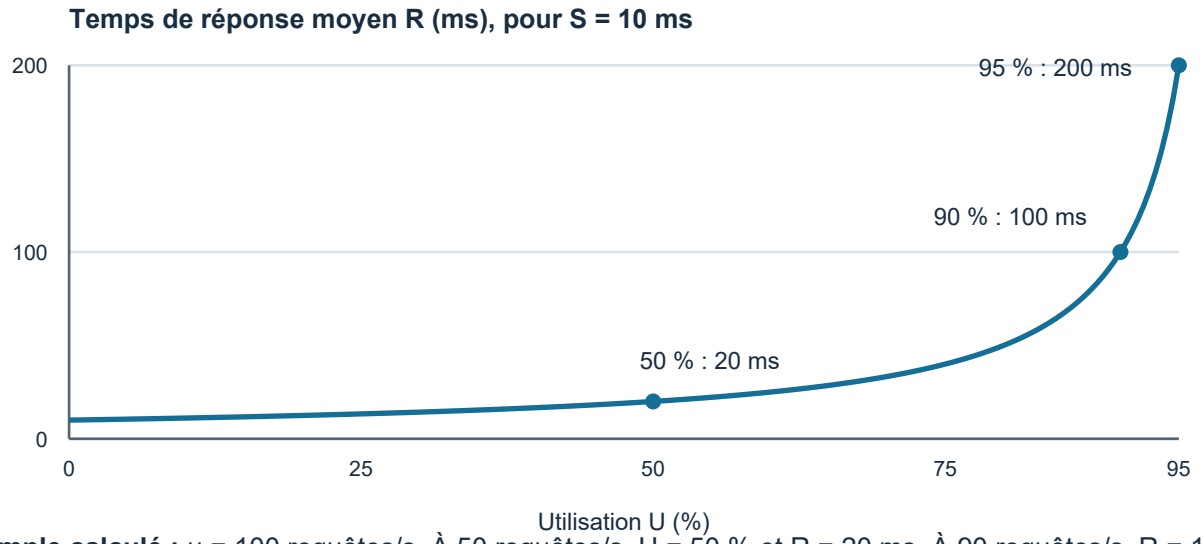
Support du cours : pages 15 à 18 • Courbe calculée à partir du modèle du cours

La scalabilité a trois dimensions : **taille** (plus d'utilisateurs), **géographie** (plus de distance) et **administration** (plus d'organisations et de règles). Ajouter des serveurs ne résout pas automatiquement les trois.

Une file M/M/1 explique le ralentissement

Hypothèses du modèle : arrivées de Poisson, durées de service exponentielles, un serveur, file illimitée et régime stationnaire avec **λ < μ**. λ désigne les requêtes par seconde ; μ la capacité moyenne de service ; S = 1/μ le temps moyen de traitement.

U = λ/μ • R = S/(1 − U) • N = U/(1 − U)
R inclut attente et traitement ; N compte les requêtes en attente et en cours. Ces moyennes ne prédisent pas tous les temps réels.



Exemple calculé : μ = 100 requêtes/s. À 50 requêtes/s, U = 50 % et R = 20 ms. À 90 requêtes/s, R = 100 ms. À 99 requêtes/s, R = 1 seconde : le traitement reste de 10 ms, mais l'attente domine.

Nuance par rapport à la page 17 : quand U tend vers 1, l'attente théorique diverge. À λ ≥ μ, il n'existe plus de régime stationnaire stable dans ce modèle ; cela ne signifie pas un arrêt instantané du serveur.

Les leviers à distinguer

Asynchronisme : poursuivre un autre travail pendant l'attente, sans annuler la latence réseau. **Partitionnement :** partager les données ou les calculs. **Réplication :** copier pour servir plusieurs endroits ou résister aux pannes. **Cache :** conserver une copie proche pour éviter un accès coûteux.

Partitionner A–M / N–Z répartit des données différentes. Répliquer A–Z sur deux nœuds crée deux copies. Les copies imposent un choix de fraîcheur et de coordination : un compteur de vues et un stock à vendre n'ont pas les mêmes exigences.

Relier le cloud à ces principes

Support du cours : pages 9 à 10, 21 à 22 et 38 • Compléments pédagogiques signalés

Le cloud fournit à distance des ressources de calcul, de stockage et des services. Son fonctionnement repose largement sur des systèmes distribués. L'utilisateur peut voir une offre unique alors que plusieurs composants coopèrent derrière elle.

Les quatre couches du support

Applications	Logiciels utilisés : bureautique, outils métier
Plateforme	Environnement et services de développement
Infrastructure	Serveurs virtuels, réseau et stockage
Matériel	Processeurs, mémoire, disques et réseau physique

Complément de vocabulaire : IaaS expose surtout l'infrastructure, PaaS un environnement géré pour exécuter une application, SaaS une application prête à utiliser. Cette grille décrit le niveau de service acheté ; elle n'est pas une équivalence parfaite entre tous les produits et les quatre couches du dessin.

Cloud et edge

Le **cloud** centralise des ressources dans des centres de données ; l'**edge** place une partie du traitement près de la source. Exemple : une caméra filtre localement des images, puis envoie des événements au cloud. On réduit certains délais et transferts, tout en ajoutant des équipements à exploiter.

Un traitement local peut aider à limiter la circulation des données sensibles ; la proximité seule ne garantit pas la sécurité.

Migrer demande de regarder les flux

L'intérêt économique dépend des ressources économisées **et** des communications supplémentaires. Pour chaque composant i, le support note $f_i = n_i/N_i$, la part migrée. Sous son hypothèse de répartition indépendante et uniforme du trafic, la fraction des échanges i → j qui traverse la frontière local/cloud vaut :

$$(1 - f_i)f_j + f_i(1 - f_j)$$

Exemple calculé : $f_i = 0{,}5$ et $f_j = 0{,}5$ donnent 0,5, soit la moitié des échanges. Migrer deux services qui dialoguent beaucoup peut coûter cher s'ils restent séparés. Il faut aussi respecter les contraintes de latence, les politiques d'hébergement et le bon déroulement des transactions.

À retenir : un déplacement dans le cloud n'améliore ni les performances ni les coûts par définition. On raisonne sur une architecture, une charge et des flux précis.

Organiser les composants

Support du cours : pages 28 à 34 • Précisions HTTP : RFC 9110, sections 9.2 et 9.3

Un style d'architecture décrit les composants, leurs connecteurs et les données échangées. Une **couche** est une séparation logique ; un **tiers** concerne le déploiement. Trois couches ne nécessitent donc pas toujours trois machines.



Style ou échange	Logique à comprendre et exemple
Couches	Chaque couche fournit un service à la suivante ; interface, métier, données.
Objets et services / RPC	Demander une action distante, par exemple calculer un prix. Un appel distant peut échouer après exécution sans que le client ait reçu la réponse.
Ressources / REST	Manipuler une ressource identifiée par une URI, avec une interface uniforme. Exemple pédagogique : GET /cours/1.
Publication et abonnement	Publier un événement sans connaître directement tous ses destinataires. Exemple : une note publiée déclenche plusieurs traitements.

REST et les méthodes HTTP

GET lit une représentation ; **PUT** crée ou remplace l'état à l'URI ciblée ; **POST** demande un traitement propre à la ressource, souvent une création dans une collection ; **DELETE** supprime l'association à la ressource ciblée. Le résumé « PUT crée / POST modifie » de la page 29 est trop restrictif.

Idempotent signifie que répéter la même demande a le même effet attendu qu'une seule : c'est le cas de PUT et DELETE, pas de POST par défaut. « Sans état » ne veut pas dire sans base de données : chaque requête doit porter le contexte nécessaire à sa compréhension.

Découplage et middleware

Référentiel : doit-on connaître le destinataire ? **Temporel :** doit-il être disponible au même moment ? Un abonnement découple les identités ; le découplage temporel nécessite aussi une conservation des messages. Tout bus d'événements n'est pas automatiquement durable.

Le **middleware** fournit des fonctions communes entre application et système local. Un **wrapper** adapte une interface ; un **intercepteur** ajoute un traitement sur le chemin d'un appel, par exemple un journal ou un contrôle d'accès.

Pour N applications, le modèle de wrappers du cours compare **N(N − 1)** adaptations dirigées en liaison directe à **2N** via un broker. Le broker simplifie les connexions, mais peut devenir un point critique.

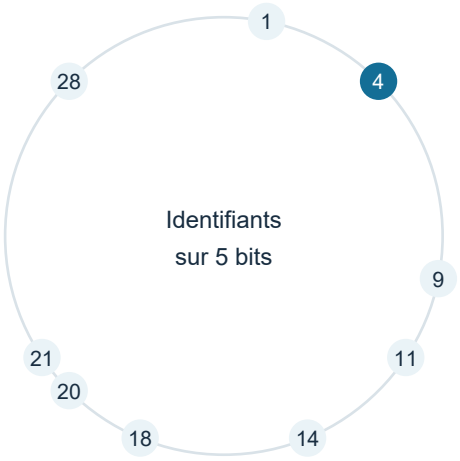
Retrouver une donnée entre pairs

Support du cours : pages 34 à 39 • Schéma original à partir des identifiants du support

En **P2P**, un processus peut être à la fois client et serveur. L'**overlay** est le graphe logique des communications entre pairs ; il ne décrit pas nécessairement les liaisons physiques du réseau.

Un réseau structuré sait vers qui chercher

Une **DHT** répartit les clés entre les nœuds. Dans Chord, une clé appartient au premier nœud rencontré dans le sens de l'anneau dont l'identifiant est supérieur ou égal à la clé, avec retour au début si nécessaire.



Clé 3 : responsable = nœud 4

Depuis le nœud 9, avec les raccourcis donnés par le cours :

9 → 28 → 1 → 4

Clé 30 : responsable = nœud 1

32 positions possibles, 9 pairs présents.

Piège de la page 35 : l'espace contient 32 identifiants possibles, mais l'exemple ne contient que 9 nœuds actifs. Les raccourcis accélèrent le routage ; ils ne changent pas le responsable de la clé.

Hypercube : deux voisins diffèrent d'un seul bit. Pour aller de 0000 à 1110, un chemin possible est 0000 → 1000 → 1100 → 1110 : un bit corrigé par saut.

Un réseau non structuré explore

Méthode	Compromis
Inondation	Diffuse aux voisins : rapide en nombre de tours, mais beaucoup de messages. Le TTL et la détection des doublons limitent l'exploration.
Marche aléatoire	Un voisin choisi à chaque étape : moins de messages par tentative, mais recherche potentiellement longue et sans succès garanti à durée bornée.

Avec N nœuds et r copies uniformément réparties, l'approximation du cours donne **E[K] ≈ N/r** sondages. Si N = 10 000 et r = 10, cela fait environ 1 000 sondages. Les visites réelles peuvent être corrélées : ce n'est pas une garantie de délai.

Pour l'inondation idéale, $\mathbf{d(d-1)^{k-1}}$ compte la frontière au niveau k, pas le cumul des niveaux. Avoir une copie attendue dans cette frontière n'assure pas de trouver l'objet avec certitude.

Autres notions du premier cours

Support du cours : pages 19 à 26, 31 à 32 et 37 à 39

Trois grandes familles de systèmes

Famille	Exemples et difficulté dominante
Calcul distribué	Cluster : machines proches et souvent homogènes. Grille : ressources hétérogènes entre organisations. Cloud : ressources proposées sous forme de services.
Systèmes d'information	Plusieurs applications coopèrent, via fichiers, base partagée, RPC ou messagerie ; formats et transactions doivent rester compatibles.
Systèmes omniprésents	Appareils mobiles, capteurs et actionneurs ; connectivité, contexte et énergie deviennent centraux.

Une transaction veut un résultat tout ou rien

Exemple pédagogique : réserver une place puis débiter un paiement. Une transaction coordonnée doit éviter de laisser une réservation confirmée avec une opération associée abandonnée. Le cours introduit BEGIN, END, ABORT, READ et WRITE, ainsi qu'un moniteur de transactions.

« Tout ou rien » désigne l'**atomicité**. En présence de pannes, plusieurs participants doivent encore coordonner leur décision.

Linda coordonne grâce à des tuples partagés

out(t) ajoute un tuple ; **rd(t)** en lit une copie ; **in(t)** le retire. Les opérations in et rd attendent un tuple correspondant s'il est absent. Deux out identiques créent deux occurrences : l'espace est un multiensemble, pas un ensemble ordinaire.

Les capteurs économisent leur batterie

Le cycle de service vaut **τ = T_actif / (T_actif + T_sommeil)**. Exemple calculé : actif 1 seconde puis endormi 9 secondes donne τ = 10 %. Les capteurs économisent de l'énergie, mais doivent avoir des périodes compatibles pour communiquer.

Les architectures hybrides mélangent les rôles

Super-pairs : certains pairs assurent davantage d'indexation ou de routage. La recherche devient plus efficace, mais leur disponibilité compte davantage. **BitTorrent dans le modèle du support :** un tracker aide à découvrir les pairs, puis les pairs échangent les morceaux du fichier ; une DHT peut aussi aider à la découverte.

Skype et les tailles de morceaux cités par le support sont des exemples historiques ou particuliers, pas des caractéristiques universelles.

Les hypothèses dangereuses à repérer

Réseau fiable, sécurisé ou homogène ; topologie fixe ; latence nulle ; bande passante infinie ; transferts gratuits ; administrateur unique : huit hypothèses à remettre en question.

Vérifier que tu as compris

Questions de révision proposées • Réponses attendues en quelques phrases

Question	Réponse courte attendue
1 Plusieurs serveurs impliquent-ils une décentralisation ?	Non. Le contrôle peut rester centralisé alors que calcul et données sont répartis.
2 Un timeout prouve-t-il une panne ?	Non. Le serveur, le réseau ou la réponse peuvent être lents ou indisponibles.
3 Réplication ou partitionnement ?	La réplication crée des copies ; le partitionnement répartit des parties distinctes.
4 S = 10 ms et U = 80 % : combien vaut R ?	R = 10/(1 − 0,8) = 50 ms, dont 40 ms d'attente moyenne dans ce modèle.
5 Pourquoi une copie peut-elle être ancienne ?	Une mise à jour n'a pas encore été propagée ou rendue visible sur cette copie.
6 Trois couches imposent-elles trois machines ?	Non. Logique applicative et placement physique sont deux décisions distinctes.
7 Un bus découple-t-il toujours dans le temps ?	Non. Il faut notamment que le message soit conservé jusqu'à sa réception.
8 Qui stocke la clé 30 de l'anneau du schéma ?	Le nœud 1 : il est le successeur après le passage de 31 à 0.
9 Pourquoi garder une marge de capacité ?	L'attente croît fortement à l'approche de la saturation, même avant 100 %.
10 Le cloud est-il toujours moins cher ?	Non. Ressources, transferts, exploitation et contraintes de délai entrent dans le bilan.

Sources et périmètre

[A] Olivier Grunder, Cloud et traitements distribués, Master DSICS, support portant la mention 2025-2026. Fichier local lu : lucc01-cours01-26.pdf, 42 pages. Les références de pages de la fiche renvoient à ce fichier. Les exemples chiffrés et les dessins ont été construits pour cette fiche.

[B] Notes personnelles locales. notes de cours.docx contient actuellement l'organisation des séances, des indications sur l'évaluation et une mention de TP Google Cloud à vérifier. Ces notes ne fixent pas ici un calendrier d'examen.

[C] van Steen et Tanenbaum, Distributed Systems, 4e éd., 2023. Présentation officielle ; diapositives officielles des neuf chapitres consultées pour vérifier le plan. L'aperçu page 1 résume les thèmes ; les chapitres 3 à 9 ne sont pas développés dans le PDF du professeur.

[D] IETF, RFC 9110, HTTP Semantics. Méthodes HTTP, section 9.3, et section 9.2 pour l'idempotence. Source de la précision apportée au résumé REST du cours.

Références complémentaires présentes page 41 du support : Cloud Computing: Concepts, Technology & Architecture, Pearson, 2013 ; Guillaume Plouin, Cloud et transformation digitale, Dunod, 2022.